

pa7uiclf

Ciência da Computação

Estrutura de dados - Filas e Pilhas, Conceitos,
Implementações e Aplicações

10/10/2025

Filas e Pilhas

- Compreender os conceitos de **Pilhas (Stack)** e **Filas (Queue)**
- Entender as diferenças entre **LIFO** e **FIFO**
- Implementar pilhas e filas em Java
- Aplicar essas estruturas em problemas práticos
- Analisar a complexidade das operações
- Timeline da história sobre mensageria

Princípio LIFO

Last In, First Out - Último a Entrar, Primeiro a Sair

↓ **PUSH**

30

20

10

↑ **POP**

Analogia Real:

 Pilha de pratos
Você sempre pega o
prato do topo!



Operações da Pilha

- ▶ **push(elemento)** - Insere um elemento no topo da pilha
- ▶ **pop()** - Remove e retorna o elemento do topo
- ▶ **peek()** - Retorna o elemento do topo sem removê-lo
- ▶ **isEmpty()** - Verifica se a pilha está vazia
- ▶ **size()** - Retorna o número de elementos



Complexidade: Todas as operações são $O(1)$

Implementado em Java

Usando a Classe Stack (java.util.Stack)

```
java
import java.util.Stack;

public class ExemploPilha {
    public static void main(String[] args) {
        Stack<Integer> pilha = new Stack<>();

        // Inserindo elementos
        pilha.push(10);
        pilha.push(20);
        pilha.push(30);

        System.out.println("Pilha: " + pilha);

        // Visualizando o topo
        System.out.println("Topo: " + pilha.peek()); // 30

        // Removendo elementos
        System.out.println("Removido: " + pilha.pop()); // 30
        System.out.println("Removido: " + pilha.pop()); // 20

        System.out.println("Pilha após remoções: " + pilha);

        // Verificando se está vazia
        System.out.println("Está vazia? " + pilha.isEmpty()); // false
    }
}
```

<https://trinket.io/java/2dd168d9ba2b>

Implementado em Java com Array

<https://trinket.io/java/354222289ba3>

Implementação Personalizada com Array

```
java
public class PilhaArray {
    private int[] elementos;
    private int topo;
    private int capacidade;

    public PilhaArray(int tamanho) {
        capacidade = tamanho;
        elementos = new int[capacidade];
        topo = -1;
    }

    public void push(int elemento) {
        if (isFull()) {
            throw new RuntimeException("Pilha cheia!");
        }
        elementos[++topo] = elemento;
    }

    public int pop() {
        if (isEmpty()) {
            throw new RuntimeException("Pilha vazia!");
        }
        return elementos[topo--];
    }

    public int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Pilha vazia!");
        }
        return elementos[topo];
    }

    public boolean isEmpty() {
        return topo == -1;
    }

    public boolean isFull() {
        return topo == capacidade - 1;
    }

    public int size() {
        return topo + 1;
    }
}
```



Aplicações Práticas - Pilha

 Desfazer/Refazer

Editores de texto usam pilhas para armazenar ações e permitir desfazer operações

 Navegador Web

Histórico de navegação e o botão "Voltar" são implementados com pilhas

 Expressões Matemáticas

Avaliação de expressões e verificação de parênteses balanceados

 Recursão

Pilha de execução para chamadas de funções recursivas

Implementando Fila de Impressão

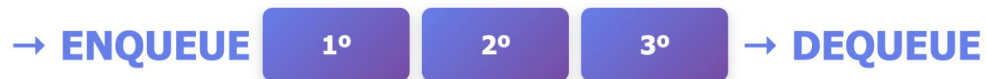
<https://trinket.io/java/5e141cef3c08>



Fila (Queue) - Conceito

Princípio FIFO

First In, First Out - Primeiro a Entrar, Primeiro a Sair



Analogia Real:



Fila de banco
Quem chega primeiro
é atendido primeiro!



Operações da Fila

- ▶ **enqueue()** / **offer()** - Insere um elemento no final da fila
- ▶ **dequeue()** / **poll()** - Remove e retorna o elemento do início
- ▶ **peek()** / **front()** - Retorna o elemento do início sem removê-lo
- ▶ **isEmpty()** - Verifica se a fila está vazia
- ▶ **size()** - Retorna o número de elementos



Complexidade: Todas as operações são $O(1)$

Implementado em Java

<https://trinket.io/java/45a84765eaa5>

Usando a Interface Queue (java.util.Queue)

```
java

import java.util.Queue;
import java.util.LinkedList;

public class ExemploFila {
    public static void main(String[] args) {
        Queue<String> fila = new LinkedList<>();

        // Inserindo elementos
        fila.offer("Cliente 1");
        fila.offer("Cliente 2");
        fila.offer("Cliente 3");

        System.out.println("Fila: " + fila);

        // Visualizando o primeiro
        System.out.println("Primeiro: " + fila.peek()); // Cliente 1

        // Removendo elementos
        System.out.println("Atendido: " + fila.poll()); // Cliente 1
        System.out.println("Atendido: " + fila.poll()); // Cliente 2

        System.out.println("Fila após atendimentos: " + fila);

        // Verificando se está vazia
        System.out.println("Está vazia? " + fila.isEmpty()); // false
    }
}
```

Implementado em Java com Estrutura Dados



<https://trinket.io/java/5dcbe94efe51>



Aplicações Práticas - Fila



Sistemas de Atendimento

Bancos, hospitais e estabelecimentos comerciais



Processos do SO

Gerenciamento de processos em sistemas operacionais



Buffer de Impressão

Fila de documentos aguardando impressão



Algoritmo BFS

Busca em largura em grafos e árvores

PriorityQueue (Fila de Prioridade)

Elementos são removidos com base em prioridade, não em ordem de chegada

<https://trinket.io/java/067a1ff00880>

Deque (Double-Ended Queue)

Permite inserção e remoção em ambas as extremidades

<https://trinket.io/java/6854394eabf7>



Pilha vs Fila - Comparação

Característica	Pilha (Stack)	Fila (Queue)
Princípio	LIFO	FIFO
Inserção	Topo	Final
Remoção	Topo	Início
Exemplo Real	Pilha de pratos	Fila de banco
Uso Principal	Desfazer, recursão	Atendimento sequencial

- ▶ **Pilhas (LIFO)** são ideais para operações que precisam desfazer ou reverter ações
- ▶ **Filas (FIFO)** são perfeitas para processamento sequencial e justo
- ▶ Ambas têm operações com **complexidade $O(1)$**
- ▶ Java oferece implementações eficientes e prontas para uso
- ▶ A escolha correta da estrutura impacta na **eficiência** do código

Timeline da Mensageria em Linguagens de Programação

Anos 70: Primórdios da Comunicação Interprocessos (IPC) no Unix

- **1970s:** Introdução de **Sinais (Signals)** e **Pipes** no Unix, permitindo comunicação básica e notificação entre processos.
- **1970s:** Desenvolvimento de **Memória Compartilhada (Shared Memory)** e **Filas de Mensagens (Message Queues)** como mecanismos mais robustos para troca de dados dentro de um sistema operacional.

Anos 80: O Nascimento da Comunicação Remota

- **Meados dos 1980s:** Conceitualização e implementação de **Chamadas de Procedimento Remoto (RPC)**, com exemplos como Sun RPC. Permitiam chamar funções em máquinas remotas como se fossem locais.

Timeline da Mensageria em Linguagens de Programação

Anos 90: Padronização e Componentes Distribuídos

- **Início dos 1990s:** O surgimento do **DCE/RPC (Distributed Computing Environment RPC)**, uma implementação mais padronizada de RPC.
- **1991:** Lançamento do **CORBA (Common Object Request Broker Architecture)**, uma ambiciosa tentativa de padronizar a interoperabilidade entre objetos em diferentes linguagens e plataformas.
- **1996:** Lançamento do **Java RMI (Remote Method Invocation)**, a solução nativa do Java para comunicação entre objetos Java distribuídos.
- **1996:** Microsoft lança **DCOM (Distributed Component Object Model)**, sua tecnologia para comunicação entre componentes Windows através da rede.
- **1998:** Introdução do **JMS (Java Message Service)**, uma API padrão Java para acessar sistemas de mensageria orientados a mensagens (MOMs), como IBM MQ e TIBCO Rendezvous.

Timeline da Mensageria em Linguagens de Programação

Anos 2000: A Ascensão dos Serviços Web

- **2000:** Padronização do **SOAP (Simple Object Access Protocol)**, marcando o início da era dos Serviços Web baseados em XML e HTTP.
- **2000s (Meados):** Popularização do estilo arquitetural **REST (Representational State Transfer)**, inicialmente como uma alternativa mais leve e simples ao SOAP, usando HTTP e formatos como JSON.
- **2006:** Lançamento do **RabbitMQ**, um dos primeiros e mais populares message brokers de código aberto, implementando o protocolo AMQP.

Timeline da Mensageria em Linguagens de Programação

Anos 2010: Escala, Streams e Microsserviços

- **2010:** Lançamento do **Apache Kafka**, inicialmente no LinkedIn, como uma plataforma distribuída de stream de eventos de alta performance, revolucionando o processamento de dados em tempo real.
- **2011:** Padronização do **WebSockets**, permitindo comunicação full-duplex persistente e em tempo real via web.
- **2015:** Lançamento do **gRPC (Google Remote Procedure Call)**, um framework RPC moderno, de alta performance, usando HTTP/2 e Protocol Buffers, ideal para microsserviços.

Anos 2020: Mensageria Ubíqua e Arquiteturas Orientadas a Eventos

- **Presente:** A mensageria (via message brokers, stream platforms e APIs REST/gRPC) é um pilar fundamental de arquiteturas de **microsserviços e arquiteturas orientadas a eventos (EDA)**, facilitando sistemas distribuídos complexos e escaláveis.
- **Presente:** Evolução contínua de padrões e ferramentas para observabilidade, segurança e gerenciamento em ambientes de mensageria distribuída.

FERRAMENAS ATUAIS DE MESSENGER BROKERS

PLATAFORMAS DE STREAMING E FILAS ROBUSTAS

Apache Kafka

Streaming de Eventos,
Alto Throughput



 High Performance
 Scalability
 Data Durability



RabbitMQ

Filas Confiáveis,
Roteamento Flexível

 Flexible Routing
 Reliable Delivery
 Multiple Work Queues



Apache ActiveMQ Artemis

Multi-Protocolo, Integração

 JMS
 Multiple Protocolo Integración
 Enterprise Integración

CLOUD MESSAGING (SERVERLESS)

 **aws SQS/SNS**

 **Google Cloud Pub/Sub**



 **Azure Service Bus**

Filas (SQS), Pub/Sub (SNS)
Escalabilidade Automática

Pub/Sub Global, Empresarial,
Baixa Latência.

MESSAGING LEVE/REAL-TIME

 **Redis Pub/Sub**



Em Memória, Rápido,
Pub/Sub Simples.

Sem Persistência

 **NATS
(JetStream)**



Alta Performance,
Microserviços
Streaming c/ JetStream

Escolha baseada em *Throughput, Confiabilidade, Flexibilidade e Escalabilidade

Algumas das mais proeminentes e suas principais características

Apache Kafka:

- a. **Foco:** Plataforma de streaming de eventos distribuída, de alta performance e tolerância a falhas. Não é apenas um broker de mensagens tradicional, mas um log de eventos distribuído e persistente.
- b. **Uso Comum:** Processamento de dados em tempo real, ingestão de logs, monitoramento de atividades, arquiteturas de microsserviços orientadas a eventos, CDC (Change Data Capture).
- c. **Vantagens:** Altíssimo throughput, escalabilidade horizontal, durabilidade dos dados (mensagens persistidas), replicação de dados, ecossistema rico (Kafka Connect, Kafka Streams, ksqldb).
- d. **Desvantagens:** Maior complexidade de setup e gerenciamento em comparação com brokers mais simples, não é ideal para cenários de "fila de trabalho" simples onde a ordem estrita de processamento não é garantida por padrão entre consumidores diferentes.

Algumas das mais proeminentes e suas principais características

RabbitMQ:

- a. **Foco:** Broker de mensagens tradicional que implementa o protocolo AMQP (Advanced Message Queuing Protocol). Conhecido por sua flexibilidade de roteamento e robustez.
- b. **Uso Comum:** Filas de trabalho (work queues), comunicação assíncrona entre microserviços, agendamento de tarefas, processamento em background.
- c. **Vantagens:** Roteamento flexível (exchanges, filas, bindings), suporte a diversos protocolos (AMQP, MQTT, STOMP), alta confiabilidade na entrega de mensagens, fácil de usar para cenários de fila de trabalho.
- d. **Desvantagens:** Menor throughput que Kafka para streaming de eventos massivo, persistência de mensagens pode impactar performance se não for bem configurada.

Algumas das mais proeminentes e suas principais características

Apache ActiveMQ (e ActiveMQ Artemis):

- A. **Foco:** Broker de mensagens multi-protocolo, robusto e flexível. ActiveMQ Artemis é a próxima geração do ActiveMQ, reescrito para melhor performance e escalabilidade.
- B. **Uso Comum:** Integração de aplicações empresariais, JMS (Java Message Service) para aplicações Java, sistemas que precisam de suporte a múltiplos protocolos.
- C. **Vantagens:** Suporte a vários protocolos (AMQP, STOMP, MQTT, OpenWire, HornetQ), alta disponibilidade, recursos de clustering.
- D. **Desvantagens:** Pode ser mais intensivo em recursos que RabbitMQ para certas cargas de trabalho, a curva de aprendizado pode ser um pouco mais íngreme para alguns dos recursos avançados.

Algumas das mais proeminentes e suas principais características

Redis (como message broker via Pub/Sub):

- a. **Foco:** Embora seja primariamente um banco de dados em memória de chave-valor, o Redis oferece um mecanismo de Publicar/Assinar (Pub/Sub) que pode ser usado para mensageria simples.
- b. **Uso Comum:** Notificações em tempo real, chat, cache de eventos, comunicação de baixa latência onde a persistência e a garantia de entrega não são a principal preocupação.
- c. **Vantagens:** Extremamente rápido (em memória), fácil de usar, já presente em muitas arquiteturas.
- d. **Desvantagens:** Não oferece persistência de mensagens (mensagens não entregues são perdidas), não há garantias de entrega, não é ideal para sistemas críticos que exigem durabilidade e reprocessamento.

Como escolher?

- **Necessidade de Streaming de Eventos e Alto Throughput: Kafka**
- **Filas de Trabalho Confiáveis e Roteamento Flexível: RabbitMQ, ActiveMQ Artemis, AWS SQS, Azure Service Bus**
- **Comunicação Pub/Sub Simples e Rápida (sem persistência crítica): Redis Pub/Sub, NATS**
- **Soluções Gerenciadas na Nuvem para Facilidade e Escalabilidade: AWS SQS/SNS, Google Cloud Pub/Sub, Azure Service Bus**
- **Microsserviços e Comunicação de Controle Leve: NATS, gRPC** (embora gRPC não seja um broker, é uma ferramenta de comunicação muito relevante).